Recognizing Smartphone Motion Gestures Using PCA and KNN

JAMES CHANG, University of Penn, USA

The proposed method is to identify single motions of the smartphones using K-nearest neighbors to classify acceleration data in X, Y, Z. Compared to the prior work in recognizing human body motions, we do not use the data of all motions to identify differences, which is not practical in interactive applications such as mobile gaming. Instead, we extract dominating base vectors from the subspace of each motion's covariance matrix through Principle Component Analysis (PCA) . The energy projected on the space from time series of one motion will be a metric for KNN classification. The method has been verified in Matlab enviroment based on the samples of three different motions collected from UnityRemote, a platform for mobile gaming development.

ACM Reference Format:
James Chang . 2026. Recognizing Smartphone Motion Gestures Using PCA and KNN. 1, 1 (January 2026), 4 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Recognizaing motions (human activities or mobile devices) is useful in different applications such as [5]. One of the applications is for the interactive applications such as mobile gaming, where the system has to identify each single motion for characters' movement and actions. Solely rely on using buttons and scratching screens would not provide good user experience esp for ? games when characterers need to jump or take turns. It would be more intuitive to use the mobile devices to control. For this specific requirement people has to rely on the built-in accerator to track users' behaviours. In this paper we explored the method to dentify single motions of the smartphones using K-nearest neighbors (KNN) to classify acceleration data in X, Y, Z. Compared to the prior work in recognizing human body motions [3], we do not use the data of all motions to identify differences, which is not practical in interactive applications such as mobile gaming. Instead, we extract the dominating subspace of each motion's covariance matrix. The energy projected on the vector from samples of one motion will be a metric for KNN classification.

The paper is organized as follows. We first introduced the previous work on recogniazing human body activity . And we would point out the limitations for moblie applicatoins. In the following session , methodology we introduced the improved algorithm to identify singe motions. The method has been verified in Matlab enviroment based on the samples of three different motions collected from UnityRemote, a platform for mobile gaming development.

Author's Contact Information: James Chang, University of Penn , Phili, USA, maam@author.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM XXXX-XXXX/2026/1-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

2 Related Work

In [3] people used the method below to identify different human body movements [1]. The top level flow chart is shown in Figure 7. There are two main steps. The first step is to extract statistical metrics (features) from the raw data through Principle Component Analysis (PCA) [2] instead of using the raw data directly. The second step is to use KNN for classification based on features from training data. One measurement is a N by 3 matrix V recording acceration data in directions of X Y Z for one activity, where V is defined by

$$V = \left[\vec{v}_x, \vec{v}_y, \vec{v}_z\right] \tag{1}$$

where $\vec{v}_x = [v_{x,1}, \ldots, v_{x,N}]^\top$ is a time series of N points recording acceration data in X direction when there is a motion happening. We will have multiple matrices of V when we repeat each motion many times. It is more convenient to seperate three directions. We have

$$X = \left[\vec{v}_{x_1}, \ldots, \vec{v}_{x_M}\right] = \begin{bmatrix} v_{x_{1,1}} & \cdots & v_{x_{1,M}} \\ \vdots & \vdots & \vdots \\ v_{x_{1,N}} & \cdots & v_{x_{N,M}} \end{bmatrix} \tag{2}$$

where X is a N by M matrix and $\vec{v}_{x_m}$, a time series , is one of multiple samples. M is the number of sampling. Similarly we will have Y and Z for other two directions.

Three statistical metrics are extracted from time series as the coordinate for distance calculation later in KNN. Two of them, u_{x_m} and σ_{x_m}, are mean and sigma for the time series, $\vec{v}_{x_m}$. The third one, e_{x_m}, is the projection (energy) of $\vec{v}_{x_m}$ on the dominating subspace of the motion as described below. First we have the covariance matrix of X [4] defined by

$$cov(X) = \frac{1}{M}(X - \bar{X})(X - \bar{X})^\top \tag{3}$$

where $\bar{X}$ is defined by

$$\bar{X} = \begin{bmatrix} v_{x_{1,1}} - u_{x,t,1} & \cdots & v_{x_{1,M}} - u_{x,t,1} \\ \vdots & \vdots & \vdots \\ v_{x_{1,N}} - u_{x,t,N} & \cdots & v_{x_{N,M}} - u_{x,t,N} \end{bmatrix} = \left[\vec{v}_{x_1} - \vec{u}_{x,t}, \ldots, \vec{v}_{x_M} - \vec{u}_{x,t}\right] \tag{4}$$

and $u_{x,t,n}$ is the mean of M sampling points at the n_{th} point in the time series defined by

$$u_{x,t,n} = \frac{1}{M}\sum_{m=1}^{M} v_{x_{n,m}} \tag{5}$$

Please note that we add t to differentiate u_{x_m} and $v_{x_{n,m}}$ is one element in [2]. $-\bar{X}$ is to centerize the original time series by removing the mean. Thanks to the covariance matrix we can capture the correlation between elements in time series when we repeat the measurement many times. And the correlation would differ depending X Y Z directions as well as different motions. Figure 6 shows the differences of covariance in Z direction among different motions in our measurement. The next step is to extract the key information

from the covariance matrix. We can find the dominating direction in the covariance matrix with the help of PCA defined by

$$cov(X) = U\Sigma U^\top \tag{6}$$

where $U = [\vec{u}_1, ..., \vec{u}_N]$ is the eigenvectors for the $cov(X)$ and Σ is a diagonal matrix. Without loss of generality we define the $\vec{u}_1$ is the dominating vector, corresponding to the biggest eigenvalue in Σ. Then $\vec{u}_1$ represents the dominating direction to the motion in X coordinate. The projection on $\vec{u}_1$, defined by

$$e_{x_m} = (\vec{v}_{x_m} - \vec{u}_{x,t})^\top \vec{u}_1 \tag{7}$$

where $\vec{v}_{x,t}$ is equation 4 and e_{x_M} is the third metric along with mean and sigma of $\vec{v}_{x_m}$. Therefore we can define a function called $F(X)$ to convert matrix X into X_{KNN} for KNN classification, where $F(X)$ converts N by M matrix into a M by 3 matrix

$$F(X) = X_{KNN} = \begin{bmatrix} u_{x_1} & \sigma_{x_1} & e_{x_1} \\ \vdots & \vdots & \vdots \\ u_{x_M} & \sigma_{x_M} & e_{x_M} \end{bmatrix} \tag{8}$$

We will have $F(Y) = Y_{KNN}$ and $F(Z) = Z_{KNN}$ following the similar procedure. The coordinate of distance calculation in KNN will be $[X_{KNN}, Y_{KNN}, Z_{KNN}]$

It is worthy noting that it is required to generate similar matrices for testing data as well for prediction. We will have $X_{KNN,test}$ from $F(X_{test})$ and $[X_{KNN,test}, Y_{KNN,test}, Z_{KNN,test}]$ [3].

Application of the classification using KNN, a supervised machine learning method, would be straightforward where each row in $[X_{KNN,test}, Y_{KNN,test}, Z_{KNN,test}]$ will be classfied based on the distance to $[X_{KNN}, Y_{KNN}, Z_{KNN}]$. We used K=1 and Euclidean distance. Figure 7 summarizes the high level flow of the method.

The method above performs very well not only in the data [1, 3] but also in our own data in the following sessions. But this method is not practical for interactive applications. The reason is that all testing data are required to generate e_{x_m} since we need the covariance matrix of all motion data to get corresponding projection on dominating subspace/direction. In gaming we need to detect a single motion alone. Detection all testing data together is not helpful. To solve this challenge we proposed an improved method to implement regonization of single motioins, without relying other testing data.

3 Methodology and Experimental results

The main improvement is the different method to extract e_{x_m} in testing data. Figure 8 provides top level pipeline. Compared with Figure 7, $e_{x_m,test}$ is not from $F(X_{test})$, covariance matrix of testing data self. Instead we project $\vec{v}_{x_m,test}$ on $\vec{u}_1$ which is from training data,X, to calculate a new $e_{x_m,test}$. In other words, we obtained $e_{x_m,test}$ from the single motion self , which differs from the previous method. And at same time we have to use the mean vector $\vec{u}_{x,t}$, from training data as well. Note that $\vec{u}_{x,t}$ is from the entire testing data as well in the previous method . We have

$$e_{x_m,new} = (\vec{v}_{x_m} - \vec{u}_{x,t,train,mean})^\top \vec{u}_{1,train} \tag{9}$$

where we add $train$ to differentiate training data and testing data and $mean$ to differentiate vector u

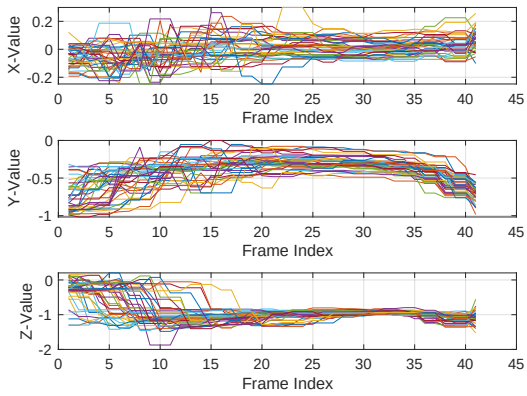


Fig. 1. Multiple measurement of acceratoin data at X, Y, Z in rotate motion

We verified the method using the data collected from UnityRemote and Matlab scripts. First we use UPDATE function and ACCELERATION class [6] in Unity/UnityRemove to capture data on X/Y/Z when we shake the phone once (Jump), rotate the phone like a wheel (Wheel) or like a key (Rotate) shown in Figure 2. We repeat 40 times for each motion. Fig 4 shows all data when Jump is repeated. Fig 1 and 5 shows all data for Wheel and Rotate . 10 percent random picked data are used for testing while the other is for training. We make sure all data are being tested. There is only one failure case shown in Figure 4 , which is not identified correctly.

4 Conclusions

The paper proposed a method to recognize single motions based on acceration data from mobile devices. The high accuracy is achieved by introduction of dominating subspace in covariance matrix of motions and usage of the projection on the subspace as the coordinate for distance calculation in KNN. Such methods would be more promising for computing due to less data size and linear operations. Implentation of such algorithms in the real applications such as interactive gaming would be the next steps.

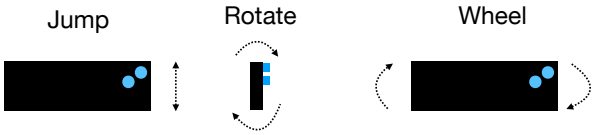


Fig. 2. Motions: Jump, Wheel and Rotate

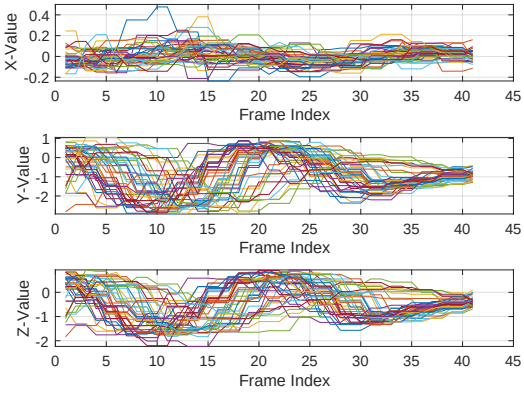


Fig. 4. Multiple measurement of acceratoin data at X, Y, Z in Jump motion

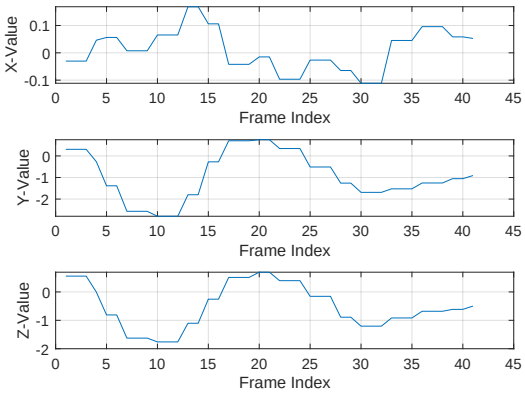


Fig. 3. one measurement of acceratoin data at X, Y, Z in Jump motion

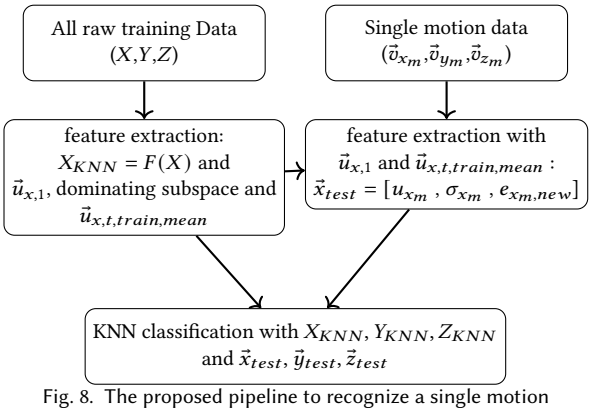


Fig. 8. The proposed pipeline to recognize a single motion

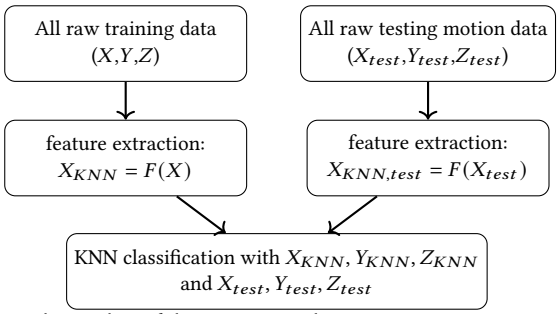


Fig. 7. The pipeline of the previous work to recognize motions together

References

[1] Luca Oneto Xavier Parra Davide Anguita, Alessandro Ghio and Jorge L. Reyes-Ortiz. 2012. Human Activity Recognition on Smartphones using a Multiclass Hardware-Friendly Support Vector Machine. *International Workshop of Ambient Assisted Living (IWAAL 2012). Vitoria-Gasteiz, Spain. Dec 2012* (2012).

[2] I. T. Jolliffe. 2002. *Principal Component Analysis (2nd ed.).* New York: Springer-Verlag.

[3] matlab. 2013. human-activity-learning-using-mobile-phone-data. https://www.mathworks.com/help/stats/human-activity-learning-using-mobile-phone-data.html. latest: https://www.mathworks.com/help/stats/human-activity-recognition-simulink-model-for-deployment.html.

[4] M. B. Priestley. 1981. *Time Series Analysis.* Academic Press.

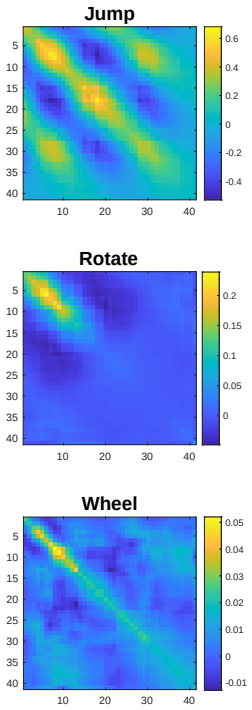


Fig. 6. Compare covariance matrix at Z direction among three motions

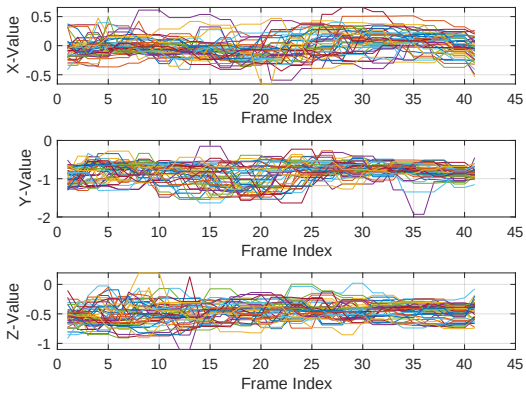


Fig. 5. Multiple measurement of acceratoin data at X, Y, Z in wheel motion

[5] Anguita Davide Ghio Alessandro Oneto Luca Reyes-Ortiz, Jorge and Xavier Parra. 2013. Human Activity Recognition Using Smartphones. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C54S4K.
[6] Unity Technologies. 2020. Unity Manual (Unity 2020.2 Documentation). https://docs.unity3d.com/2020.2/Manual/UnityManual.html. Accessed: 2025-12-26.